Aprendizaje por refuerzo para LLMs

Clase 8 · Procesamiento de Lenguaje Natural

Andrés Abeliuk, Fabrizzio Pezzolla

Diplomado en Inteligencia Artificial · DCC · Universidad de Chile

Dónde quedamos: imitar no basta

Clase 7

Con ajuste fino supervisado (SFT) y QLoRA, 1.000 pares de instrucción y respuesta convirtieron un modelo base en un asistente.

El límite del SFT

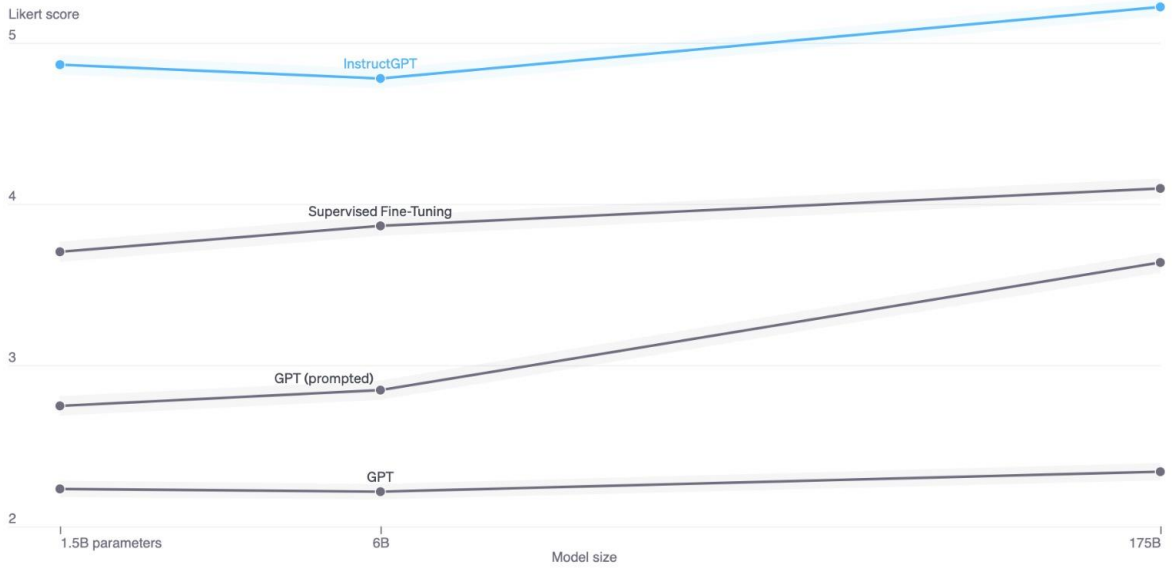
Según LIMA, el SFT enseña sobre todo el formato. Y solo imita: no sabe cuál de dos respuestas es mejor.

Hoy

Aprender de recompensas: preferencias humanas (RLHF, DPO) y verificadores automáticos (RLVR).

De imitar respuestas a optimizar una recompensa

¿Vale la pena el RLHF?



+1 punto

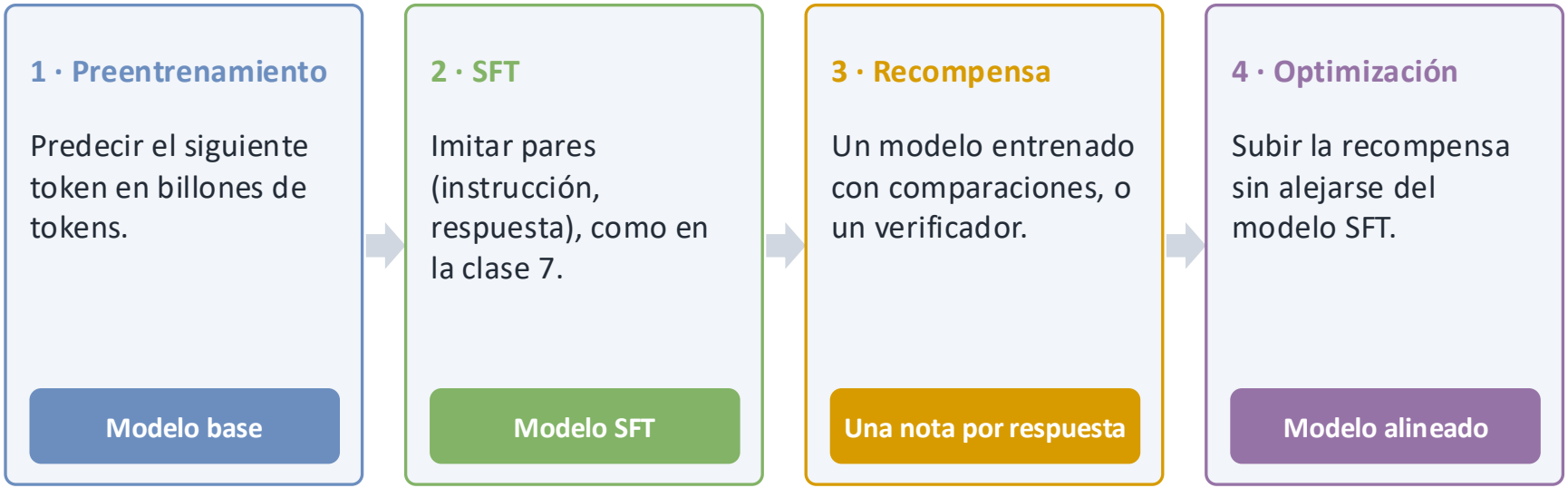
en una escala de 1 a 7: con RLHF, mejor que con SFT solo, en todos los tamaños.

1.300 millones

de parámetros con RLHF se prefieren a GPT-3 de 175.000 millones.

Respuestas a prompts reales de la API (Ouyang et al., 2022; figura vía Huyen, 2023). Según Bai et al. (2022), el feedback humano rinde más cuando juzgar es fácil y formalizar, difícil.

El pipeline de post-entrenamiento



No hay una frontera matemática entre pre- y post-entrenamiento: el post-entrenamiento se hace pensando en tareas y benchmarks concretos (Hardt, 2026).

¿Qué datos necesita cada método?

SFT (clase 7)

Pares prompt → respuesta ideal, escrita por alguien. El modelo aprende a imitarla.

La respuesta ideal

RL con preferencias

Solo prompts. El modelo genera y un modelo de recompensa le pone nota.

Modelo de recompensa

Preferencias directas

Tripletas (prompt, ganadora, perdedora). Se aprende directo de los pares.

Pares de preferencias

RL con verificador

Solo prompts. El modelo genera y un programa revisa la respuesta.

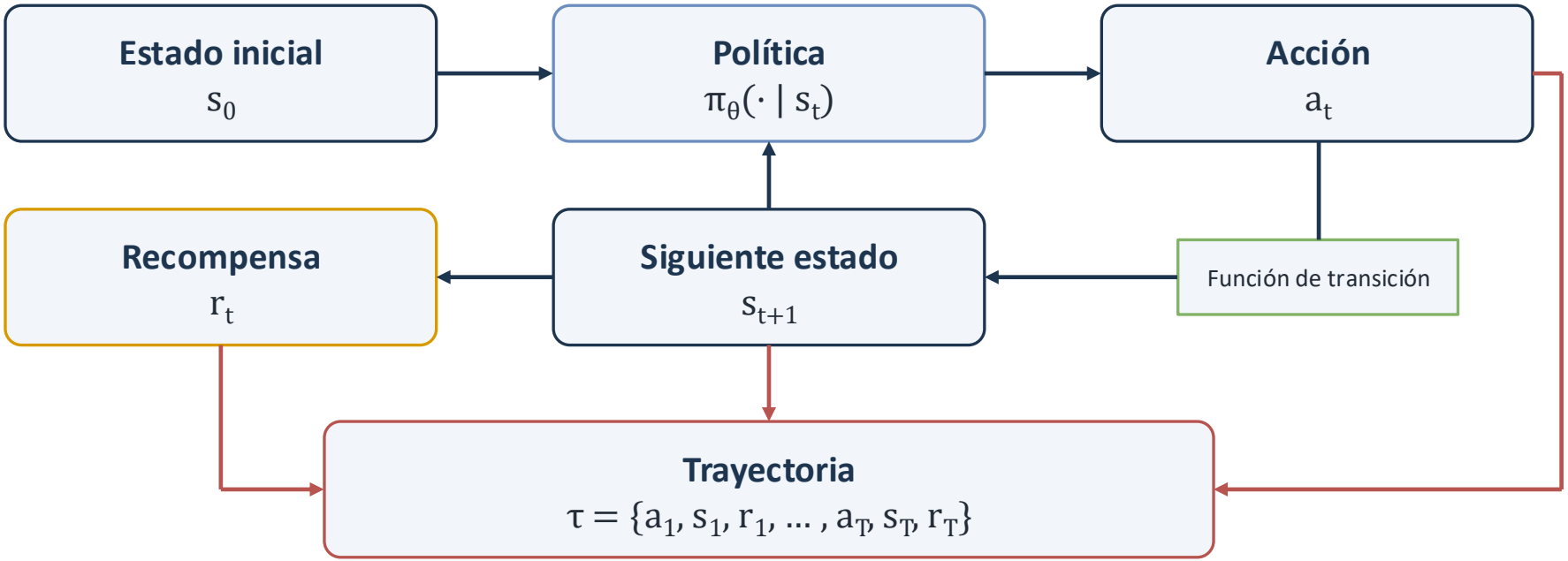
Verificador

SFT necesita la respuesta correcta. Los demás solo necesitan saber qué respuesta es mejor

Aprendizaje por refuerzo

Lo justo para entender PPO, DPO y GRPO

¿Qué es el aprendizaje por refuerzo?



Adaptado de Wolfe (2025), PPO for LLMs.

Un LLM es una política

¿Hacen envíos a regiones?

Sí	,	enviamos	a	todo	Chile	<eos>
r = 0	r = 0	r = 0	r = 0	r = 0	r = 0	**r = 0,8**

Política	El LLM que entrenamos (π_θ)
Estado inicial	El prompt
Acción	El siguiente token
Estado	El prompt + los tokens generados hasta ahora
Recompensa	Un verificador o un modelo de recompensa: solo al final

Adaptado de Wolfe (2025), PPO for LLMs.

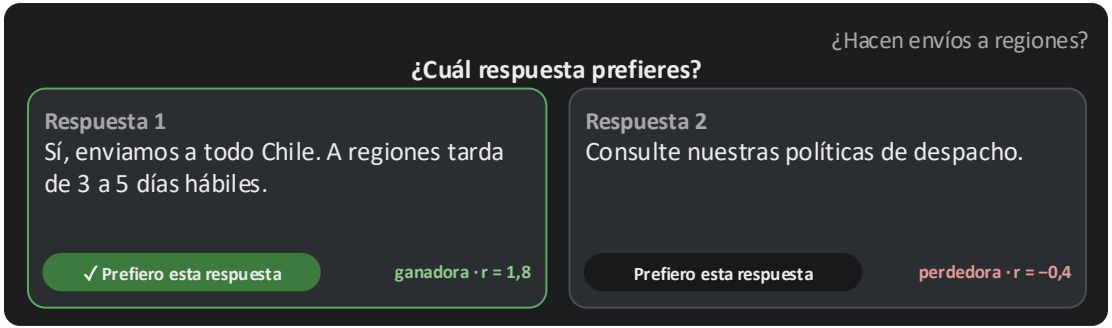
El modelo de recompensa: aprender r(x, y)

Objetivo: una función r(x, y) que le ponga nota a cualquier respuesta y a un prompt x.

prompt + respuesta → LLM (copia del SFT) → cabeza lineal en el último token → **una nota r(x, y)**

Se entrena con comparaciones

Una persona ve dos respuestas al mismo prompt y elige la que prefiere, como en ChatGPT. La elegida es la ganadora del par; la otra, la perdedora.



Aprende a darle más nota a la ganadora que a la perdedora. Esa r(x, y) será la recompensa del RL

Policy gradient: la idea general

Objetivo: la recompensa esperada
$J(\theta) = E[\, r(x, y) \,]$

Buscamos los mejores parámetros
$\theta^* = \text{argmax}_\theta \, J(\theta)$

Solución: ascenso de gradiente
$\theta_{k+1} = \theta_k + \alpha \, \nabla_\theta \, J(\theta)$

$$\nabla_\theta J(\theta) = \mathbb{E}_\tau \left[\sum_t \nabla_\theta \log \pi_\theta (a_t \mid s_t) \cdot \Psi_t(r) \right]$$

1 · Generar
El modelo muestrea respuestas: son las trayectorias τ de la fórmula.

2 · Pesar
Cada token recibe un peso Ψ, calculado con la recompensa r. El más simple: Ψ = r(x, y).

3 · Actualizar
Como el SFT, pero con pesos: refuerza si Ψ > 0, debilita si Ψ < 0.

Es como el SFT sobre las respuestas del propio modelo, pero cada token pesa según la recompensa

Cómo se calcula Ψ a partir de la recompensa es lo único que cambia entre algoritmos: siguiente diapositiva.

Instancias: ¿qué peso Ψ usa cada algoritmo?

$\Psi_t = R(\tau)$	**REINFORCE:** Un peso por respuesta: su recompensa, R(τ) = r(x, y), igual para todos sus tokens. Simple, pero muy ruidoso.
$\Psi_t = R(\tau) - b$	**REINFORCE con línea base:** Un peso por respuesta, menos lo esperado b: un número fijo, como el promedio del lote. Mucho menos ruido.
$\Psi_t = A(s_t, a_t)$	**PPO:** Un peso por token: la ventaja A = Q − V. Lo esperado en ese punto, V(s), lo estima un crítico entrenado aparte.
$\Psi_i = (r_i - \text{promedio}) \,/\, \text{desv}$	**GRPO:** Un peso por respuesta, relativo a un grupo de respuestas al mismo prompt. Sin crítico.

Problema de REINFORCE: si todas las recompensas son positivas, todo sube. ¿Qué respuesta fue realmente buena? → ventaja

Dos diferencias entre filas: el peso va por respuesta o por token, y lo esperado es un promedio o lo estima un crítico.

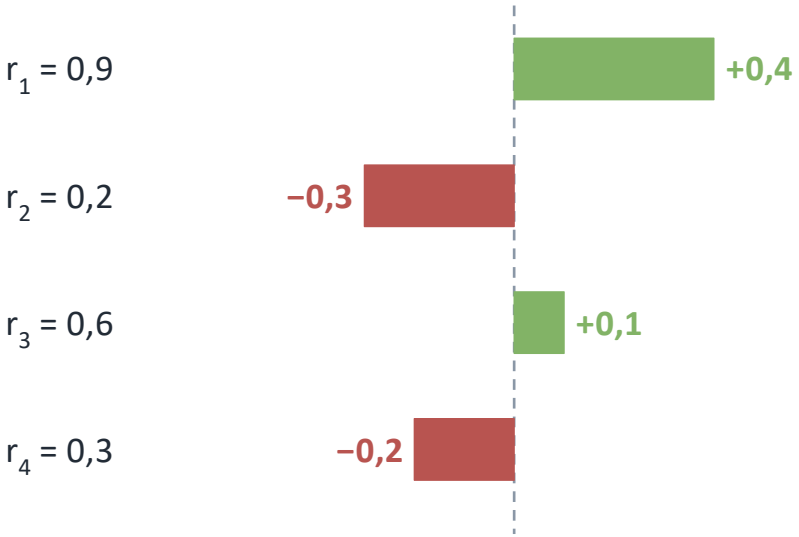
Ventaja: ¿mejor o peor de lo esperado?

El problema: 4 respuestas al mismo prompt, todas con recompensa positiva. Con Ψ = R, las cuatro suben, también las malas. **La solución:** restar lo esperado.

$$A(s,a) = Q(s,a) - V(s)$$

- **Q y V:** Q es lo que obtuvo la respuesta; V, lo que esperábamos (la línea base). Aquí V es el promedio: 0,5.
- **A > 0:** fue mejor de lo esperado → subir su probabilidad.
- **A < 0:** fue peor → bajarla.
- **Restar una línea base** no cambia el gradiente esperado, solo reduce el ruido.

Con línea base: r − 0,5



¿Quién estima V(s)? En PPO, un crítico: otro LLM entrenado. En GRPO, el promedio de un grupo de respuestas.

Reward hacking

Verbosidad

Respuestas cada vez más largas, porque el modelo de recompensa aprendió que lo largo suele ganar.

Adulación

Darle la razón a la persona aunque esté equivocada: a la gente le gusta que le den la razón.

Trucos raros

Frases, formatos o emojis que suben r sin mejorar la respuesta.

Ley de Goodhart: cuando una medida se vuelve objetivo, deja de ser una buena medida

Remedios: no alejarse del modelo SFT (siguiente diapositiva), mejores modelos de recompensa y revisar muestras a mano.

El freno: no alejarse de la referencia

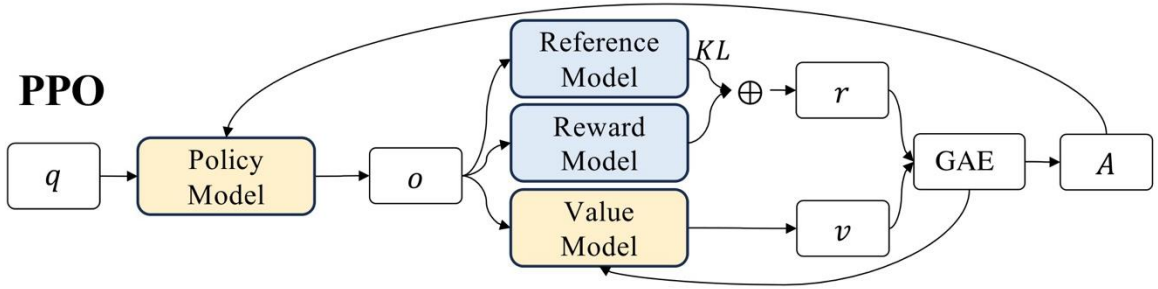
Para que el modelo no explote las fallas de la recompensa, se penaliza que se aleje del modelo SFT:

$$\max_{\theta} \mathbb{E}_{x,\, y\sim\pi_\theta}[r(x,y)] - \beta\, \mathrm{KL}(\pi_\theta(y \mid x) \;\|\; \pi_{\mathrm{ref}}(y \mid x))$$

- **π_{ref}:** el modelo SFT congelado. La KL mide cuánto se alejó la política de él.
- **β grande:** cambios pequeños y seguros. β chico: más libertad para subir r (y para hacer trampa).
- **En la práctica,** la KL se calcula token a token: $\beta\,(\log \pi_\theta - \log \pi_{ref})$.

Este es el objetivo que optimizan PPO y GRPO, y del que se deriva DPO

PPO: la receta clásica (y cara)



Cómo leerla

q: prompt · o: respuesta · r: nota · A: ventaja.
Value Model: el crítico. Su salida v es V(s), lo esperado.
Amarillo: se entrena. **Azul:** congelado.

- **El crítico estima V(s):** es lo que se resta en la ventaja, A(s, a) = Q(s, a) − V(s).
  En la figura, v. Es un segundo LLM que hay que entrenar.
- **Recorte:** en cada paso, la probabilidad de un token cambia a lo más ±20 % (ε = 0,2).
- **Cuatro LLM en memoria:** es caro de entrenar.

Por eso surgieron alternativas sin crítico (GRPO) o directamente sin RL (DPO)

Figura 4 de Shao et al. (2024), DeepSeekMath (parte superior).

De PPO a DPO y GRPO: qué se ahorra cada uno

PPO

Cuatro LLM en memoria: política, referencia, modelo de recompensa y crítico. Funciona, pero es caro.

DPO

Sin modelo de recompensa y sin generar respuestas: aprende directo de pares (ganadora, perdedora), como un clasificador.

GRPO

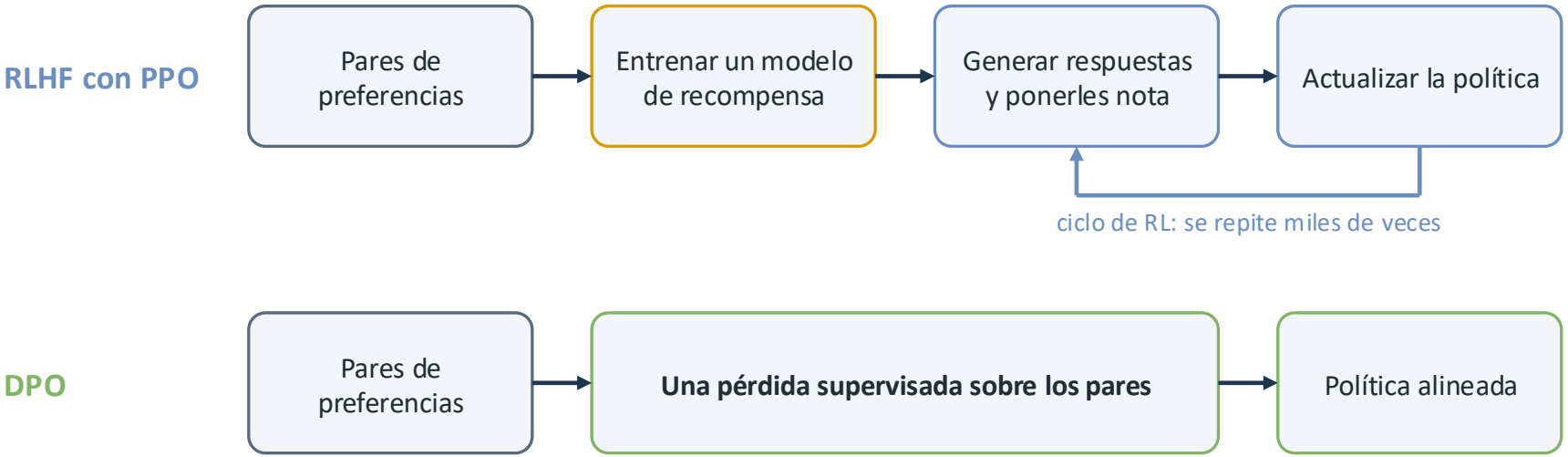
Sin crítico: la línea base es el promedio de un grupo de respuestas al mismo prompt. Entrenó a DeepSeek-R1.

La pregunta que más decide: ¿un programa puede revisar la respuesta? Si puede, un verificador reemplaza al modelo de recompensa

Sin aprendizaje por refuerzo: DPO

Optimizar directamente sobre las comparaciones

RLHF vs. DPO: qué se elimina



DPO elimina el modelo de recompensa y el ciclo de RL: queda un entrenamiento supervisado

Adaptado de Rafailov et al. (2023), Direct Preference Optimization, y de Wolfe (2025).

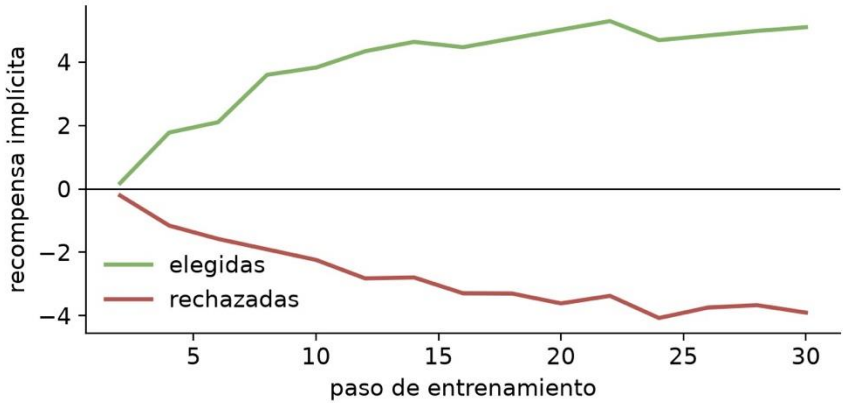
Qué aprende DPO con cada par

Para cada par: ¿el modelo hace más probable la elegida que la referencia? ¿Y la rechazada?

`x: ¿Hacen envíos a regiones?`

Elegida ↑ más probable
"Sí, enviamos a todo Chile. A regiones tarda de 3 a 5 días hábiles."

Rechazada ↓ menos probable
"Consulte nuestras políticas."

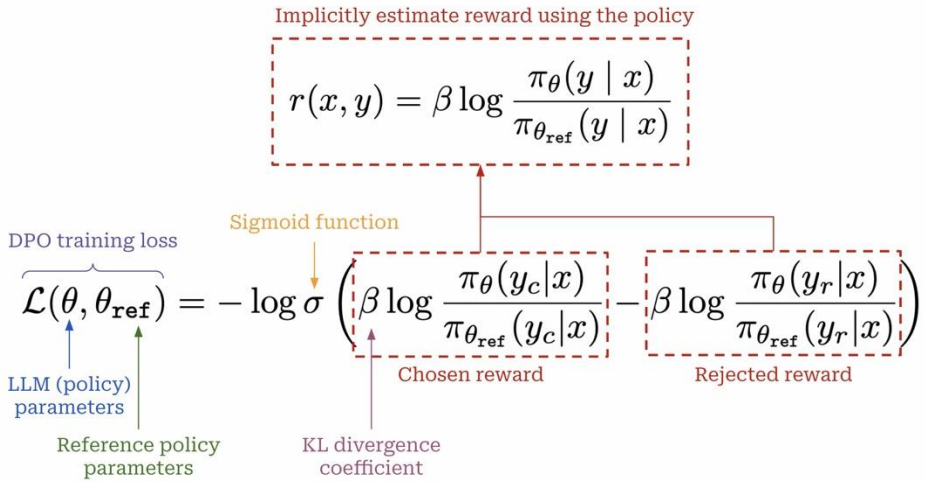


Del laboratorio: DPOTrainer, 60 pares, 30 pasos.

Sube la elegida y baja la rechazada, siempre medido respecto de la referencia

No genera respuestas ni consulta un modelo de recompensa: solo mira los pares. Por eso no explora respuestas nuevas.

La pérdida de DPO, leída por partes



Recompensa implícita
Cuánto más probable hace el modelo una respuesta que la referencia.

Elegida menos rechazada
La nota de la elegida (chosen) tiene que superar a la de la rechazada (rejected).

–log σ(diferencia)
La pérdida del modelo de recompensa: pesa más los pares que aún ordena mal.

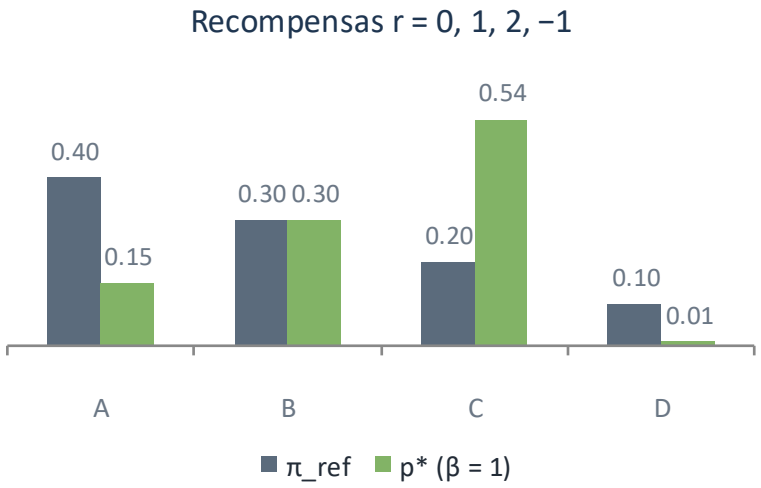
El propio LLM hace de modelo de recompensa

Figura de Wolfe (2025), Direct Preference Optimization. Rafailov et al. (2023), Your Language Model is Secretly a Reward Model.

¿De dónde sale? Despejar la recompensa

$$p^*(y \mid x) = \frac{1}{Z(x)}\,\pi_{\text{ref}}(y \mid x)\exp(r(x,y)/\beta)$$

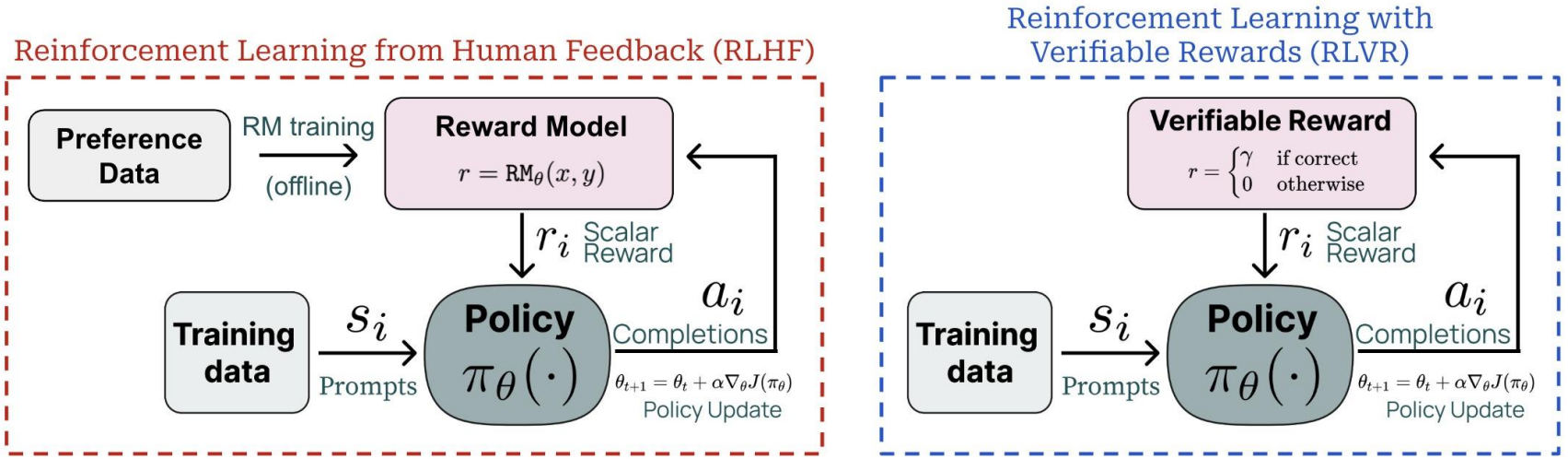
- **Con el freno KL,** la mejor política posible es la referencia reponderada: más peso a las respuestas con más recompensa.
- **El truco:** despejar r de esta fórmula. La recompensa queda escrita con el propio LLM, y Z(x) se cancela al comparar dos respuestas.



GRPO y recompensas verificables

Sin crítico y, cuando se puede, sin modelo de recompensa

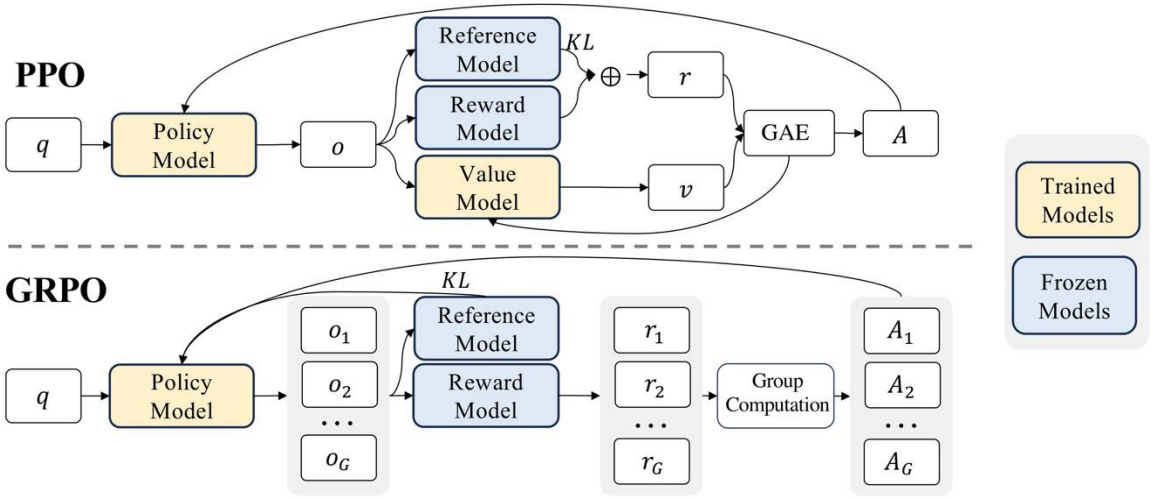
RLHF vs. RLVR: de dónde sale la recompensa



RLHF: la nota la pone un modelo entrenado con preferencias. RLVR: una regla que revisa la respuesta

Figura de Wolfe (2025), PPO for LLMs, a partir de Lambert et al. (2024), Tülu 3.

PPO vs. GRPO: qué cambia



Cómo leerla
q: el prompt · o: la respuesta · r: la nota · A: la ventaja.

PPO
Entrena dos modelos (en amarillo): la política y el crítico (Value Model).

GRPO
G respuestas por prompt y un cálculo de grupo en vez del crítico.

GRPO es PPO sin crítico: lo esperado es el promedio de las otras respuestas al mismo prompt

Figura 4 de Shao et al. (2024), DeepSeekMath. La nota puede venir de un verificador o de un modelo de recompensa: eso no depende de GRPO.

GRPO: la ventaja relativa al grupo

$$\hat{A}_i = \frac{r_i - \text{promedio}(r_1, \ldots, r_G)}{\text{desv}(r_1, \ldots, r_G)}$$

10 cuadernos a $1.200 con oferta 3×2: ¿cuánto se paga?

y_1: $8.400 ✓	r = 1	**Â = +0,87**
y_2: $12.000 ✗	r = 0	**Â = −0,87**
y_3: $8.400 ✓	r = 1	**Â = +0,87**
y_4: $9.600 ✗	r = 0	**Â = −0,87**

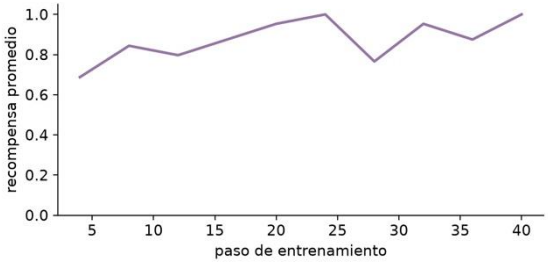
- **Sin crítico:** la línea base es el promedio del grupo. Aquí: promedio 0,5 y desviación 0,58 (con n − 1, como TRL), así que (1 − 0,5) / 0,58 ≈ +0,87.
- **Todos los tokens** de una respuesta comparten su ventaja.
- **Conserva** el recorte de PPO y la KL con π_{ref}.
- **Con un verificador,** solo dos LLM en memoria: política y referencia.

Shao et al. (2024), DeepSeekMath · DeepSeek-AI (2025), DeepSeek-R1.

GRPO en el laboratorio: ¿qué grupos enseñan?

Sumas de dos números, 8 respuestas por problema. El verificador pone 1 si el número es exacto y 0 si no.

solución	respuestas del grupo	ventaja
82	72 72 **82** 72 72 72 72 72	la correcta: +2,5
73	73 73 53 73 73 73 73 73	la incorrecta: −2,5
57	47 37 47 37 47 47 47 47	todas mal: sin señal
79	79 79 79 79 79 79 79 79	todas bien: sin señal



47 % → 92 %

de acierto con una muestra, tras 40 pasos

Solo los grupos mezclados enseñan: el problema tiene que ser difícil, pero no imposible

Del laboratorio: GRPOTrainer sobre Qwen2.5-0.5B con LoRA. Nadie le mostró una respuesta correcta; solo la nota del verificador.

Cuando la respuesta se puede verificar

Matemática
Comparar el número final con la solución.
`"82" == "82"  →  r = 1`

Código
Ejecutar el programa y pasar las pruebas.
`3 de 3 pruebas  →  r = 1`

Formato
¿Es un JSON válido, con las claves pedidas?
`{"producto": …}  →  r = 1`

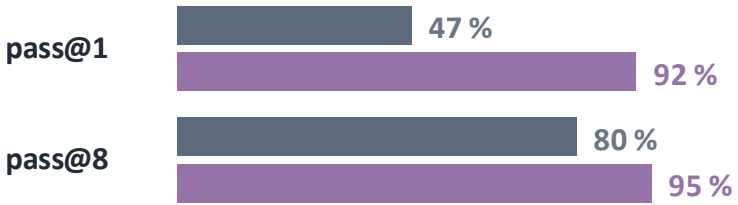
pass@k: ¿acierta al menos una de k respuestas?

Solución: 76

76	36	86	86	27	86	27	86

Con una respuesta, acierta 1 de cada 8 veces. Con las 8, al menos una acierta: quedarse con una correcta de N (best-of-N) ya es una mejora.

En el laboratorio: antes y después de GRPO



RLVR sube mucho el acierto con una sola respuesta: enseña a usar lo que el modelo ya sabía generar

Yue et al. (2025): con k grande, el modelo base alcanza al entrenado con RLVR. Así se entrenan los modelos de razonamiento, como o1 y DeepSeek-R1.

DeepSeek-R1-Zero: RL desde el modelo base

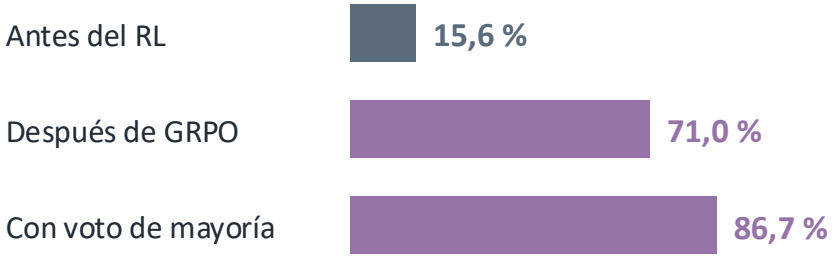
La recompensa: dos reglas, ningún modelo

Exactitud
¿La respuesta final coincide con la solución?

Formato
¿El razonamiento va entre <think> y </think>?

Parte del modelo base (DeepSeek-V3-Base), sin SFT previo, y se entrena solo con GRPO.

AIME 2024 (prueba de matemática de EE. UU.)

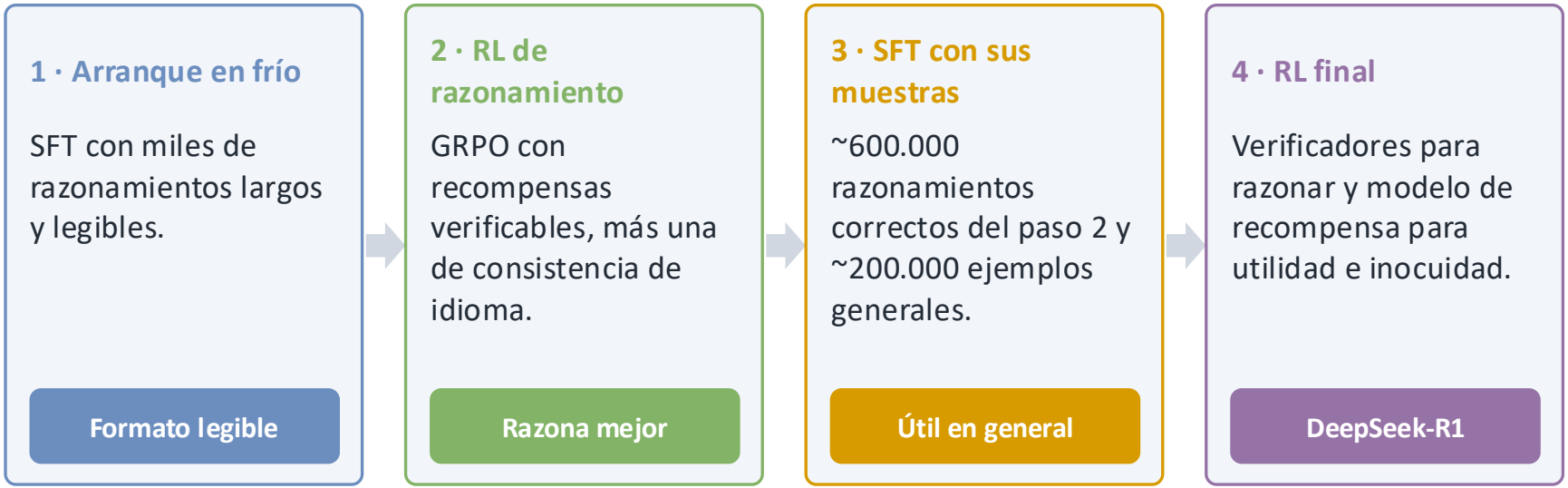


Aprende solo a pensar más: sus respuestas se alargan y aparece la autocorrección (“Wait, wait… That’s an aha moment”).

Nadie le mostró cómo razonar: lo descubrió con GRPO y un verificador

Pero sus razonamientos son poco legibles y mezclan idiomas: lo corrige DeepSeek-R1 (siguiente diapositiva). DeepSeek-AI (2025).

DeepSeek-R1: la receta completa



Destilación: con esos 800.000 ejemplos se ajustan modelos Qwen y Llama de 1,5B a 70B; el de 7B llega a 55,5 % en AIME 2024 (DeepSeek-AI, 2025).

RLVR con un solo ejemplo

El experimento

Qwen2.5-Math-1.5B entrenado con GRPO sobre un único problema de matemática, repetido en cada lote.

El resultado

En MATH500 pasa de 36,0 % a 73,6 %: lo mismo que entrenar con unos 1.200 problemas.

Por qué funciona

La capacidad ya estaba en el modelo preentrenado. Sigue mejorando en prueba aun después de memorizar el ejemplo.

Como en LIMA (clase 7): RLVR no enseña conocimiento nuevo, enseña a usar el que el modelo ya tiene

Wang et al. (2025). Funciona también con PPO y en otros modelos (Llama, DeepSeek-R1-Distill); fomentar la exploración es clave.

Comparación

	Señal	LLM en memoria	¿Genera al entrenar?	Uso típico
PPO	Modelo de recompensa (preferencias)	4: política, referencia, crítico, recompensa	Sí	RLHF clásico: InstructGPT, ChatGPT
DPO	Pares de preferencias	2: política, referencia	No	Alinear modelos abiertos: Zephyr, Llama 3, Tülu 3
GRPO	Verificador (o modelo de recompensa)	2–3: política, referencia (+ recompensa)	Sí, G por prompt	Razonamiento: DeepSeek-R1

Resumen

1. Comparar es más fácil que escribir: con pares (ganadora, perdedora) se entrena un modelo de recompensa.
2. En RL el LLM es una política: sube las respuestas con ventaja positiva, sin alejarse de la referencia (KL).
3. PPO funciona, pero necesita cuatro modelos: política, referencia, recompensa y crítico.
4. DPO elimina el modelo de recompensa y el RL: se entrena como un clasificador sobre pares.
5. GRPO elimina el crítico con la ventaja de grupo; con un verificador, sobra el modelo de recompensa.

Ahora en Python

1. Bradley-Terry: de comparaciones a recompensas
2. Un modelo de recompensa con RewardTrainer
3. DPO con DPOTrainer
4. La recompensa implícita de Qwen
5. GRPO con GRPOTrainer y un verificador
6. RLHF con GRPO: reward hacking y el freno KL

Todo sobre un LLM de verdad: Qwen2.5-0.5B con LoRA, entrenado con TRL. En Colab, con GPU T4.